Faculty of Engineering

Topics in Computer Science
Local consistency

PhD. Willy Ugarte Rojas
willyugarte@gmail.com

Constraint Satisfaction Problem

- A constraint satisfaction problem (CSP) is a tuple (X, D, C) where:
  - $X = \{x_1, x_2, \ldots, x_n\}$ is the set of variables
  - $D = \{d_1, d_2, \ldots, d_n\}$ is the set of domains
    (d_i is a finite set of potential values for x_i)
  - $C = \{c_1, c_2, \ldots, c_m\}$ is a set of constraints

- For example: $x, y, z \in \{0, 1\}, x + y = z$ is a CSP where:
  - Variables are: x, y, z
  - Domains are: $d_x = d_y = d_z = \{0, 1\}$
  - There is a single constraint: $x + y = z$

Constraints

- A constraint C is a pair (S, R) where:
  - $S = (x_{i_1}, ..., x_{i_k})$ are the variables of C (scope)
  - $R \subseteq d_{i_1} \times ... \times d_{i_k}$ are the tuples satisfying C (relation)
- According to this definition: $x + y = z$ in the CSP $x, y, z \in \{0, 1\}, x + y = z$ is short for

$$((x, y, z), \{(0, 0, 0), (1, 0, 1), (0, 1, 1)\})$$

- A tuple $\tau \in d_{i_1} \times ... \times d_{i_k}$ satisfies C iff $\tau \in R$
- The arity of a constraint is the size of its scope
  - Arity 1: unary constraint (usually embedded in domains)
  - Arity 2: binary constraint
  - Arity 3: ternary constraint
  - ...
- This corresponds to the extensional representation of constraints

Constraints

- But constraints are usually described more compactly: intensional representation
- A constraint with scope S is determined by a function

$$\prod_{x_i \in S} d_i \longrightarrow \{\text{true}, \text{false}\}$$

- Satisfying tuples are exactly those that give true
- In the example: $x + y = z$
- Unless otherwise stated, we will assume that evaluating a constraint takes time linear in the arity
- This is usually, but not always, true

Solution

- Given a CSP with variables $X = \{x_1, x_2, \ldots, x_n\}$,
  domains $D = \{d_1, d_2, \ldots, d_n\}$ and constraints C,
  a solution is an assignment of values $(x_1 \mapsto \nu_1, \cdots, x_n \mapsto \nu_n)$ such that:
  - Domains are respected: $\nu_i \in d_i$
  - The assignment satisfies all constraints in C
- Solving a CSP consists in finding a solution to it
- Other related problems:
  - Finding all solutions
  - Finding a best solution wrt. an objective function
    (then we talk of a Constraint Optimization Problem)

Examples (I): Prop. Satisfiability

- Given a formula F in propositional logic, is F satisfiable?
- Variables are the atoms of the formula
- Variables have all domain $\{\text{true}, \text{false}\}$
- A single constraint: the evaluation of F must be 1

- Let F be $(p \vee q) \ \wedge \ (p \vee \neg q) \ \wedge \ (\neg p \vee q)$:

Examples (I): Prop. Satisfiability

- Given a formula F in propositional logic, is F satisfiable?
- Variables are the atoms of the formula
- Variables have all domain $\{\text{true}, \text{false}\}$
- A single constraint: the evaluation of F must be 1

- Let F be $(p \vee q) \ \wedge \ (p \vee \neg q) \ \wedge \ (\neg p \vee q)$:
  - Variables are p, q

Examples (I): Prop. Satisfiability

- Given a formula F in propositional logic, is F satisfiable?
- Variables are the atoms of the formula
- Variables have all domain $\{\text{true}, \text{false}\}$
- A single constraint: the evaluation of F must be 1

- Let F be $(p \vee q) \ \wedge \ (p \vee \neg q) \ \wedge \ (\neg p \vee q)$:
  - Variables are p, q
  - Domains are $d_p = d_q = \{\text{true}, \text{false}\}$

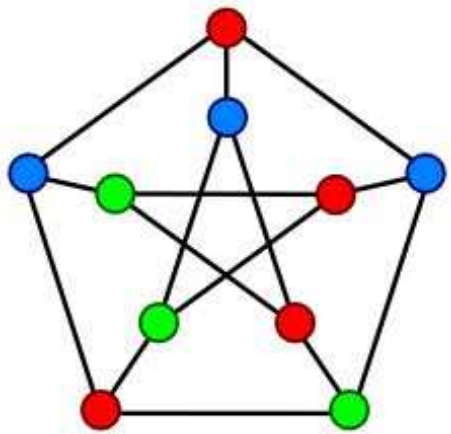
Examples (I): Prop. Satisfiability

- Given a formula F in propositional logic, is F satisfiable?
- Variables are the atoms of the formula
- Variables have all domain $\{\text{true}, \text{false}\}$
- A single constraint: the evaluation of F must be 1

- Let F be $(p \vee q) \ \wedge \ (p \vee \neg q) \ \wedge \ (\neg p \vee q)$:
  - Variables are p, q
  - Domains are $d_p = d_q = \{\text{true}, \text{false}\}$
  - Constraint is $(p \vee q) \ \wedge \ (p \vee \neg q) \ \wedge \ (\neg p \vee q) = \text{true}$

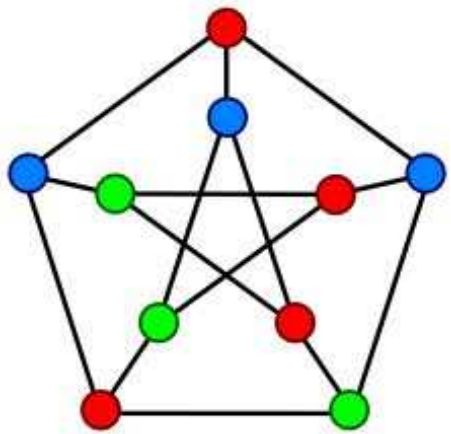
Examples (II): Graph Coloring

- Given a graph $G = (V, E)$ and $K > 0$ colors,
  can vertices be painted so that neighbors have different colors?



Examples (II): Graph Coloring

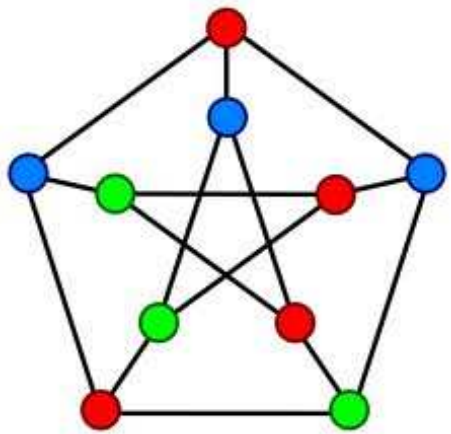
- Given a graph $G = (V, E)$ and $K > 0$ colors,
  can vertices be painted so that neighbors have different colors?



  - Variables are $\{c_v \mid v \in V\}$, the color for each vertex

Examples (II): Graph Coloring

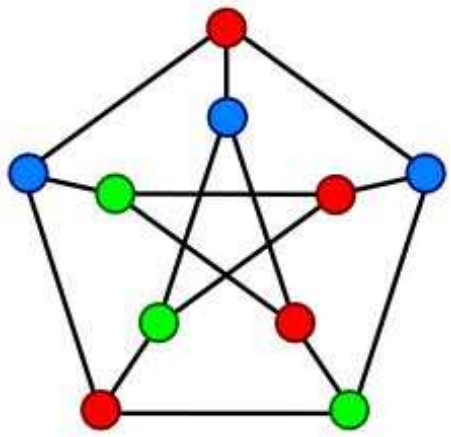
- Given a graph $G = (V, E)$ and $K > 0$ colors,
  can vertices be painted so that neighbors have different colors?



  - Variables are $\{c_v \mid v \in V\}$, the color for each vertex
  - Domains are $\{1, 2, \ldots, K\}$, the available colors

Examples (II): Graph Coloring

- Given a graph $G = (V, E)$ and $K > 0$ colors,
  can vertices be painted so that neighbors have different colors?

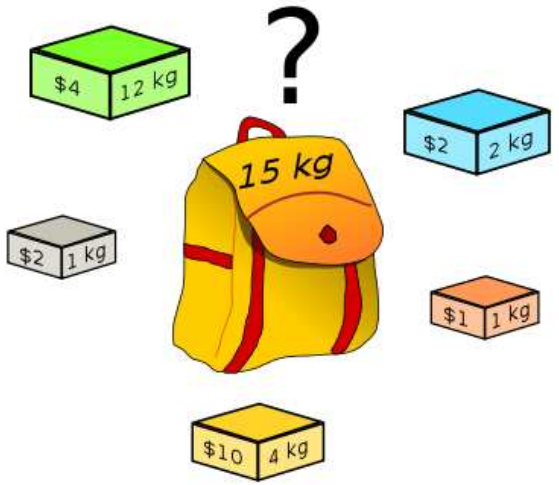


  - Variables are $\{c_v \mid v \in V\}$, the color for each vertex
  - Domains are $\{1, 2, \ldots, K\}$, the available colors
  - Constraints are: for each $(u, v) \in E$, $c_u \neq c_v$

Examples (III): Knapsack

- Given:
  - n items with weights w_i and values v_i
  - a capacity W
  - a number V,

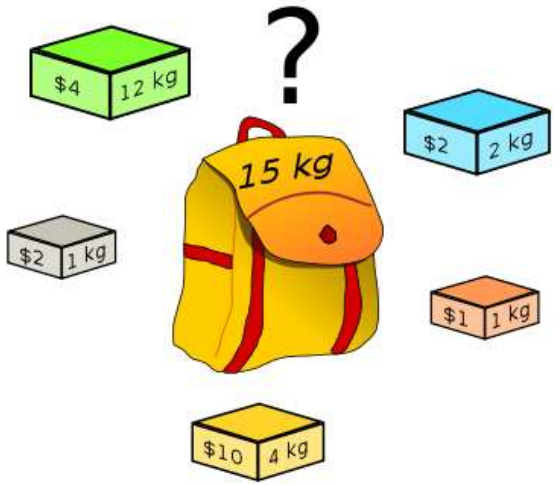
  is there a subset S of the items s.t. $\sum_{i \in S} w_i \leq W$ and $\sum_{i \in S} v_i \geq V$?



Examples (III): Knapsack

- Given:
  - n items with weights w_i and values v_i
  - a capacity W
  - a number V,

  is there a subset S of the items s.t. $\sum_{i \in S} w_i \leq W$ and $\sum_{i \in S} v_i \geq V$?

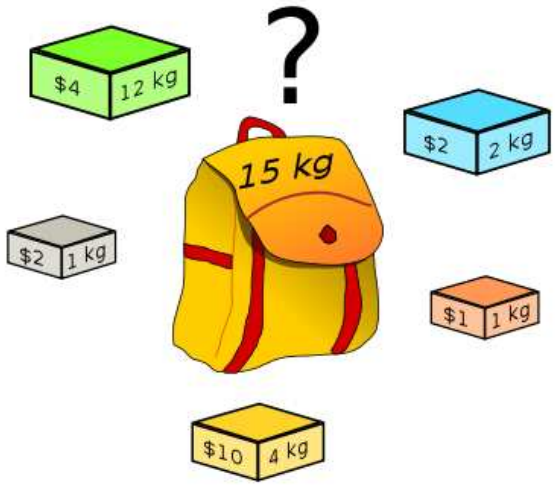


- Variables: n variables x_i meaning "item i is selected"

Examples (III): Knapsack

- Given:
  - n items with weights w_i and values v_i
  - a capacity W
  - a number V,

  is there a subset S of the items s.t. $\sum_{i \in S} w_i \leq W$ and $\sum_{i \in S} v_i \geq V$?

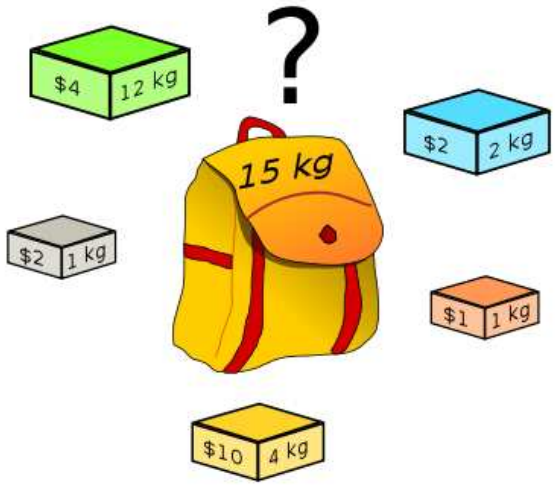


- Variables: n variables x_i meaning "item i is selected"
- Domains: $d_i = \{0, 1\}$

Examples (III): Knapsack

- Given:
  - n items with weights w_i and values v_i
  - a capacity W
  - a number V,

  is there a subset S of the items s.t. $\sum_{i \in S} w_i \leq W$ and $\sum_{i \in S} v_i \geq V$?



- Variables: n variables x_i meaning "item i is selected"
- Domains: $d_i = \{0, 1\}$
- Constraints: $\sum_{i=1}^{n} w_i x_i \leq W$, $\sum_{i=1}^{n} v_i x_i \geq V$

Complexity

- **Theorem**. Solving a CSP is an NP-complete problem

  Proof:

  - It is in NP, because one can check a solution in polynomial time
  - It is NP-hard, as there is a reduction e.g. from Prop. Satisfiability (which is known to be NP-complete)

- For any CSP, there are instances that require exp time
  Can we solve real life instances in reasonable time?

1 Previously

2 Introduction
2.1 Another Example: Graph Coloring
2.2 Variables, Domains and Constraints
2.3 Solving:
Brute Force
Backtracking

3 Constraint Programming
3.1 CSP
3.2 CP solving
3.3 History
3.4 Solvers

4 Examples

5 Applications

6 Conclusion

Solving

Generate and Test

- How can we solve CSP's?
- 1st naïf approach: Generate and Test (aka Brute Force)
  - Generate all possible candidate solutions (assignments of values from domains to variables)
  - Test whether any of these is a true solution indeed

Generate and Test

- Example: *Queens Problem*. Given $n \geq 4$,
  put n queens on an $n \times n$ chessboard so that they don't attack each other

  Wlog, we can place one queen per row
  so that no two are in the same column or diagonal.

  - Variables: c_i, column of the queen of row i
  - Domains: all domains are $\{1, 2, \ldots, n\}$
  - Constraints: no two are in same column/diagonal

Q				
Q				
Q				
Q				
Q				

Q				
Q				
Q				
Q				
	Q			

Basic Backtracking

- Generate and Test is very inefficient
- 2nd approach to solving CSP's: Basic Backtracking
- The algorithm maintains a partial assignment that is consistent with the constraints whose variables are all assigned:
  - Start with an empty assignment
  - At each step choose a var and a value in its domain
  - Whenever we detect a partial assignment that cannot be extended to a solution, backtrack: undo last decision

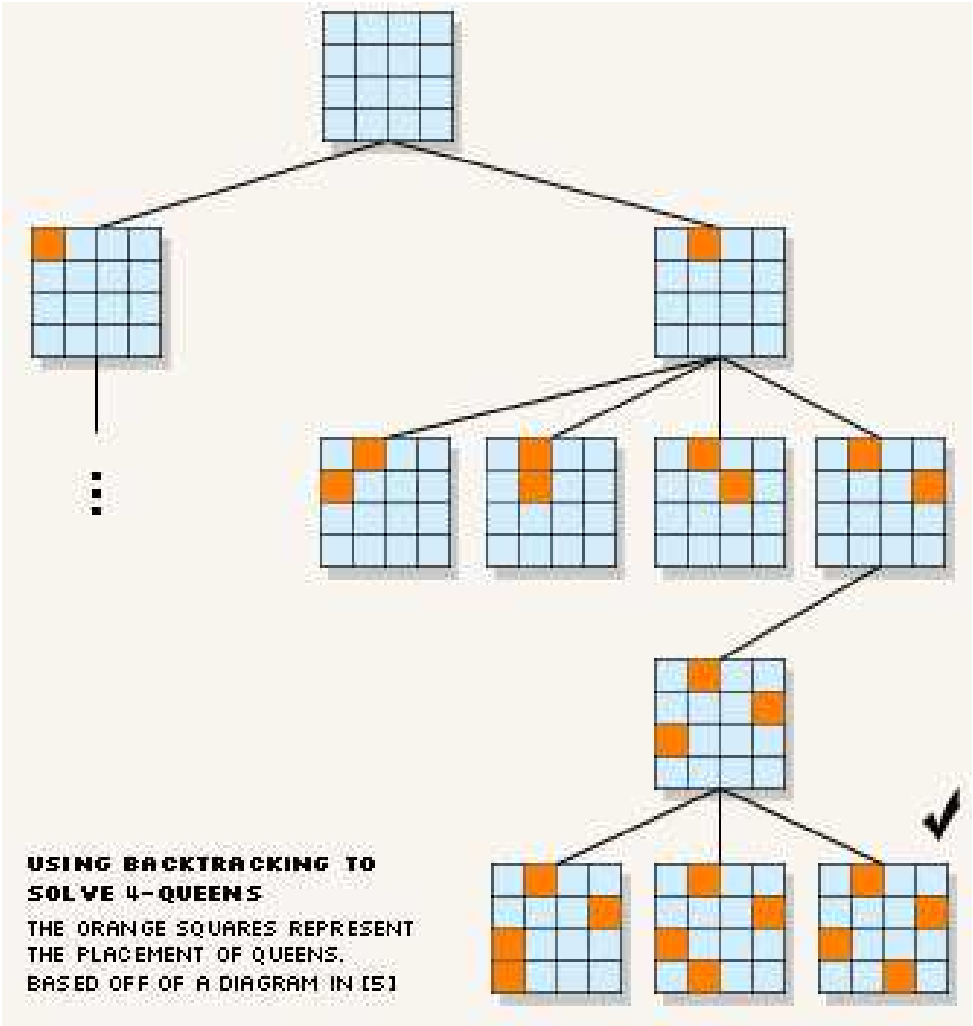
Basic Backtracking

- We can solve the problem by calling `backtrack(x1)`:

```
function backtrack(variable X) returns bool
  for all a in domain(X) do
    val(X) := a
    if compatible(X, assigned)
      assigned := assigned ∪ {X}
      if no next(X)                    then return TRUE
      else if backtrack(next(X)) then return TRUE
      else assigned := assigned - {X}
  return FALSE

function compatible(variable X, set A) returns bool
  for all constraint C with scope in A∪{X} and not in A do
     // Let A be {Y1, ..., Yn}
     if (val(X), val(Y1),..., val(Yn)) don't satisfy C then
        return FALSE
  return TRUE
```

Basic Backtracking



Basic Backtracking

- The set of all possible partial assignments forms a search tree:
  - The root corresponds to the empty assignment
  - Each edge corresponds to assigning a value to a var
  - For each node, there are as many children as values in the domain of the chosen variable
  - Generate and Test corresponds to visiting each of the leaves until a solution is found
  - Complexity: $O(m^n \cdot e \cdot r)$
    - $n =$ no. of variables
    - $m =$ size of the largest domain
    - $e =$ no. of constraints
    - $r =$ largest arity
  - Basic Backtracking performs a depth-first traversal
  - Complexity: the same, as in the worst case we need to visit all leaves
  - But in practice it works much better than Generate and Test

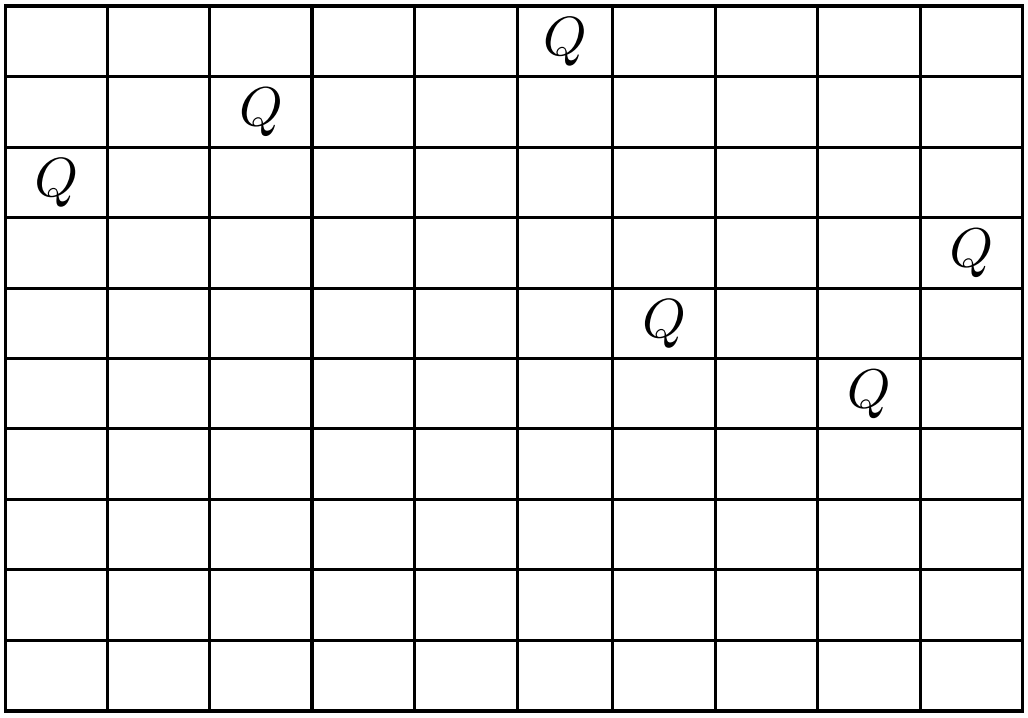
Basic Backtracking

- Problems with backtracking
  - Inconsistencies may be found late, after a lot of useless work

    If $x_1 \mapsto a$ is incompatible with $x_n \mapsto \text{anything}$,
    then BT explores the subtree rooted at $x_1 \mapsto a$
    (if x_1 can take m values, this subtree is $\frac{1}{m}$ of the whole search tree!)
    to realize that no solution can be found

  - The right backtracking point may not be the last decision

Basic Backtracking



Basic Backtracking

					Q				
		Q		X	X	X			
Q	X	X	X		X		X		
X	X	X		X	X			X	Q
X	X	X			X	Q		X	X
X		X	X		X	X	X	Q	X
X		X		X	X	X	X	X	X
X		X	X		X	X		X	X
X		X		X	X	X		X	X
X	X	X	X	X	X	X	X	X	X

Propagation

- CP approach: prune search tree a priori by removing values from the domains that can't appear in any solution

```
while (solution not found) do
    assign values to some of the variables
    propagate with constraints to prune other domains
    if (found inconsistency) undo last decision
```

- Smaller search tree, at the cost of more time per node
- There exist different kinds of propagation with different tradeoffs between pruning power and cost in time

Propagation

Q				
X	X			
X		X		
X			X	
X				X

Propagation

Q				
X	X	Q		
X	X	X	X	
X		X	X	X
X		X		X

Propagation

Q				
X	X	Q		
X	X	X	X	Q
X		X	X	X
X		X		X

Propagation

Q				
X	X	Q		
X	X	X	X	Q
X	Q	X	X	X
X	X	X		X

Propagation

Q				
X	X	Q		
X	X	X	X	Q
X	Q	X	X	X
X	X	X	Q	X

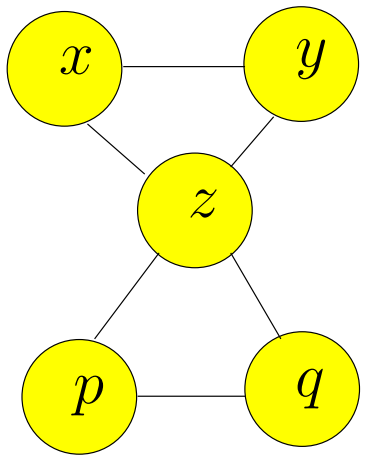
No search!

Consistency

Interaction Graph

- The interaction graph of a CSP is an undirected graph $G = (V, E)$ s.t.:
  - there is a vertex i associated to each variable x_i
  - there is an edge (i, j) if
    there exists some constraint having both x_i and x_j in its scope

CSP with Boolean variables x, y, z, p, q
and constraints: $x + y = z, |p - q| = z$



- For example, the interaction graph of graph coloring is the same graph!

Binary CSP's

- A CSP is binary if all its constraints have arity 2
- When considering binary CSP's,
  c_{ij} indicates the constraint between variables x_i and x_j
- In what follows we will focus on binary CSP's.
  We can do it wlog because of the following property:

  Theorem. Any CSP can be transformed into an equisatisfiable binary one.

Binary CSP's

- Consider the CSP with Boolean variables x, y, z, p, q and constraints: $x + y = z, |p - q| = z$.
- The equivalent binary CSP has:
  - *variables*: one for each original constraint

$$v_{x+y=z}, \qquad v_{|p-q|=z}$$

  - *domains*: the tuples that satisfy the original constraint

$$\begin{aligned} d_{x+y=z} &= \{(x,y,z) \in \{(0,0,0),(1,0,1),(0,1,1)\}\} \\ d_{|p-q|=z} &= \{(p,q,z) \in \{(0,0,0),(1,0,1),(0,1,1),(1,1,0)\}\} \end{aligned}$$

  - *constraint*: satisfying tuples are consistent on common values

$$\{((0,0,\mathbf{0}),(0,0,\mathbf{0})), \quad ((0,0,\mathbf{0}),(1,1,\mathbf{0})),$$
$$((1,0,\mathbf{1}),(1,0,\mathbf{1})), \quad ((1,0,\mathbf{1}),(0,1,\mathbf{1})),$$

Filtering, Propagation

- Let $P = (X, D, C)$ be a (binary) CSP,
  $x_i \in X$ a variable and $a \in d_i$ a domain value
- $P[x_i \to a]$ is the CSP obtained from P by replacing d_i by $\{a\}$
- $a \in d_i$ is feasible if $P[x_i \to a]$ has solutions, unfeasible otherwise
- Example:
  - Let x, y be two integer variables with domains $[1, 10]$
  - Consider constraint $|x - y| > 5 \equiv (x - y > 5) \vee (y - x > 5)$
  - Values 5 and 6 for both variables are unfeasible

Filtering, Propagation

- Let $P = (X, D, C)$ be a (binary) CSP,
  $x_i \in X$ a variable and $a \in d_i$ a domain value
- $P[x_i \to a]$ is the CSP obtained from P by replacing d_i by $\{a\}$
- $a \in d_i$ is feasible if $P[x_i \to a]$ has solutions, unfeasible otherwise
- Example:
  - Let x, y be two integer variables with domains $[1, 10]$
  - Consider constraint $|x - y| > 5 \equiv (x - y > 5) \vee (y - x > 5)$
  - Values 5 and 6 for both variables are unfeasible
- Detecting unfeasible values is useful because
  they can be removed without losing solutions
- In general, detecting if a value is feasible is an NP-Complete problem
- But in some cases unfeasible values can be easily detected
- Filtering algorithms identify and remove unfeasible values efficiently

Local Consistency

- A clean way of designing filtering techniques is by means of the concept of local consistency
  - A local consistency property allows identifying unfeasible values: inconsistent values, i.e., not satisfying the property, are unfeasible
  - Local because only small pieces of the problem are considered (typically, one constraint)
  - Enforcing a local consistency property means propagating: removing the inconsistent values until the property is achieved
  - Enforcing local consistency should be cheap (polynomial time)

Local Consistency Properties

- The most important is Arc Consistency (AC)
  (aka Domain Consistency)
- Weaker than AC:
  - Directional AC (DAC)
  - Bounds Consistency (BC)
  - ...
- Stronger than AC:
  - Singleton AC (SAC)
  - Neighborhood Inverse Consistency (NIC)
  - ...

Arc Consistency

- Let $P = (X, D, C)$ be a (binary) CSP
- Value $a \in d_i$ of variable $x_i \in X$ is arc-consistent wrt. x_j if there is $b \in d_j$ (the support of a in x_j) s.t. $c_{ij}(a, b) = \text{true}$
- The definition of arc-consistency is then lifted in the natural way:
  - Variable $x_i \in X$ is arc-consistent wrt. x_j if all values in its domain are arc-consistent wrt. x_j
  - Constraint $c_{ij} \in C$ is arc-consistent if x_i is arc-consistent wrt. x_j, and vice-versa
  - A CSP P is arc-consistent if all $c \in C$ are arc-consistent
- Notation: AC means arc-consistent
- If $a \in d_i$ is arc-inconsistent (not arc-consistent) wrt. some x_j, it is unfeasible. Hence, it can be safely removed!
- Enforcing AC means removing arc-inconsistent values until AC is achieved

Arc Consistency

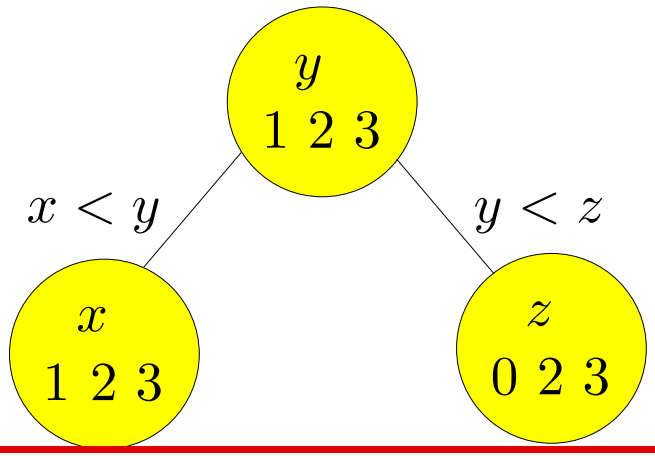
- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints

Arc Consistency

- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

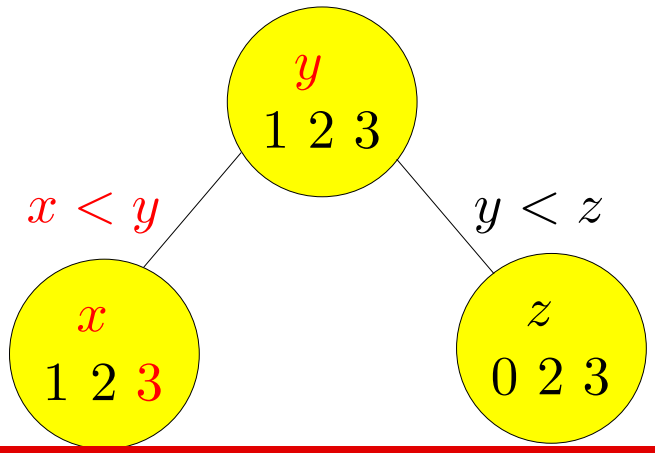
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints



Arc Consistency

- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

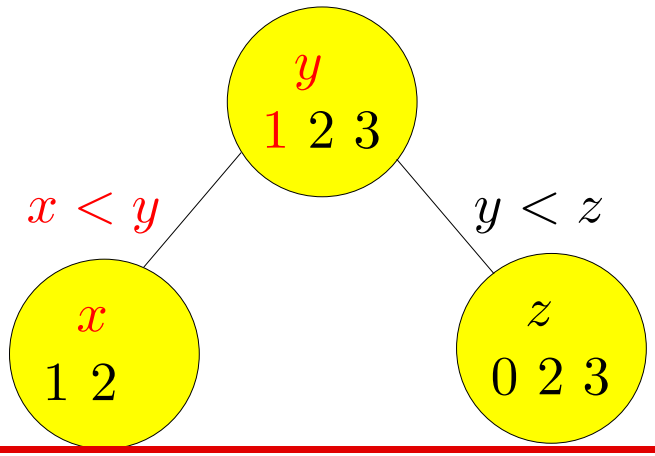
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints



Arc Consistency

- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

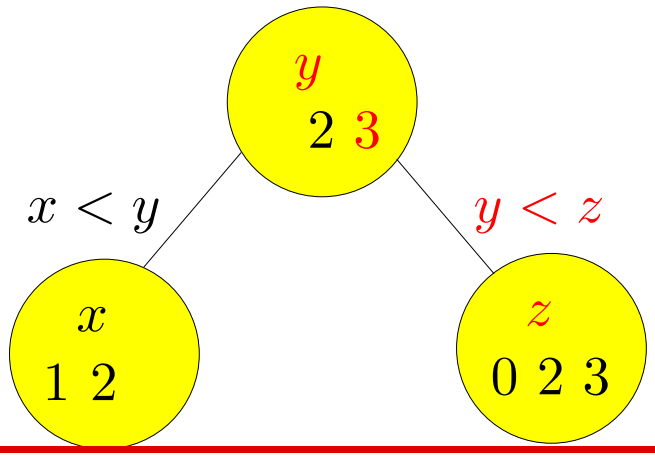
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints



Arc Consistency

- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

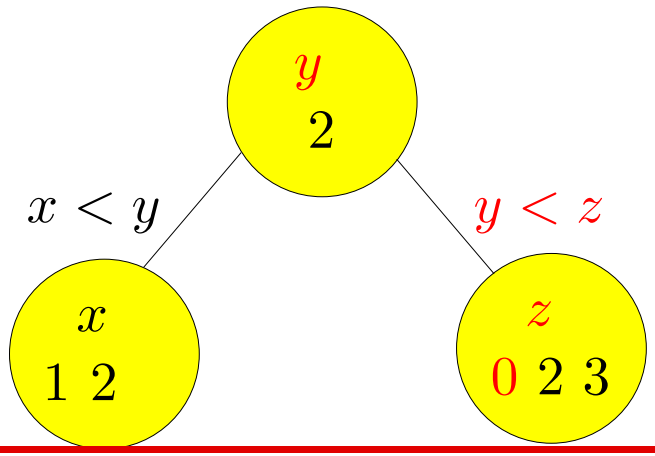
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints



Arc Consistency

- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

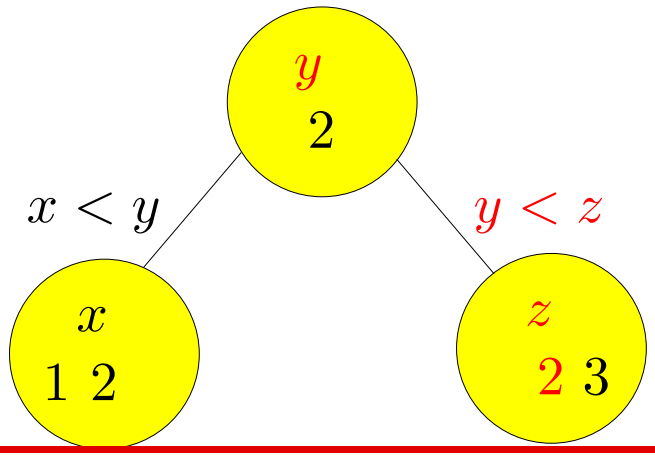
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints



Arc Consistency

- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

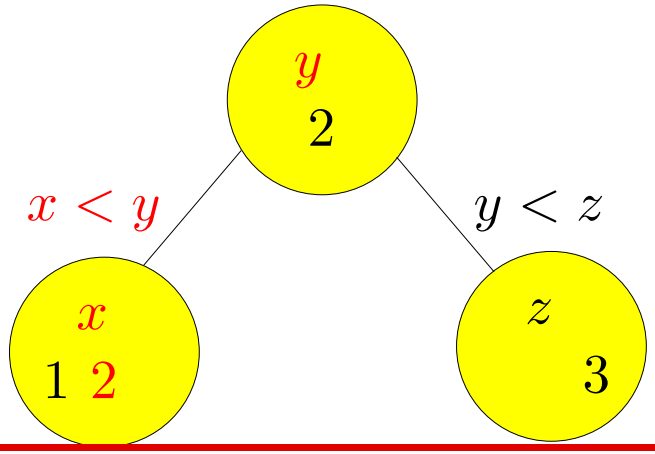
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints



Arc Consistency

- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

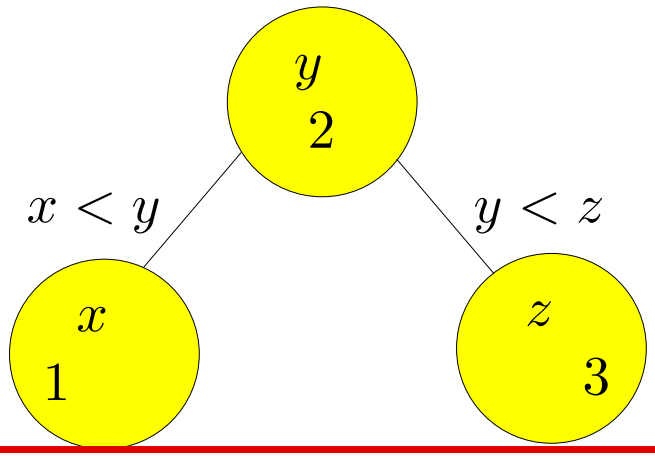
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints



Arc Consistency

- The AC name comes from the arcs (constraints) of the interaction graph
- Example of enforcing AC.

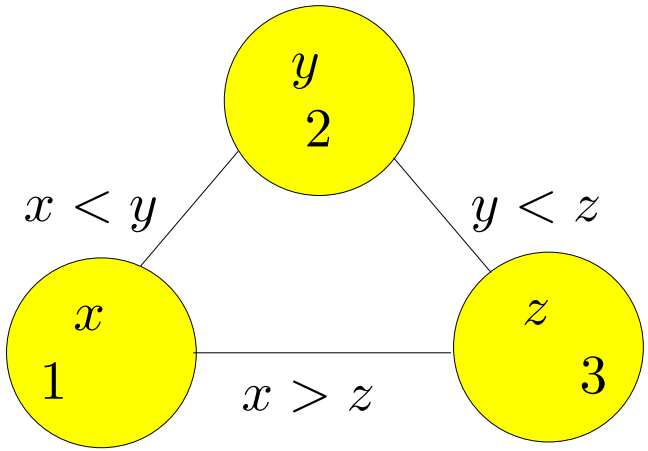
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y, \quad y < z$
- Recall nodes are labelled with variables (here, also with their domains)
- Recall edges are labelled with constraints



Arc Consistency

- Another example of enforcing AC.

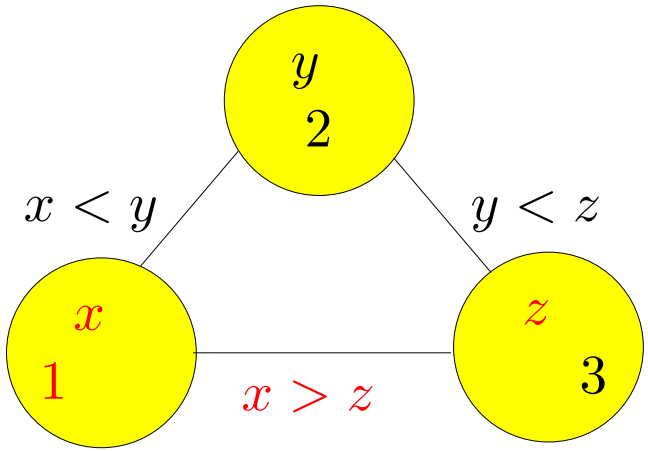
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y,\ y < z,\ x > z$



Arc Consistency

- Another example of enforcing AC.

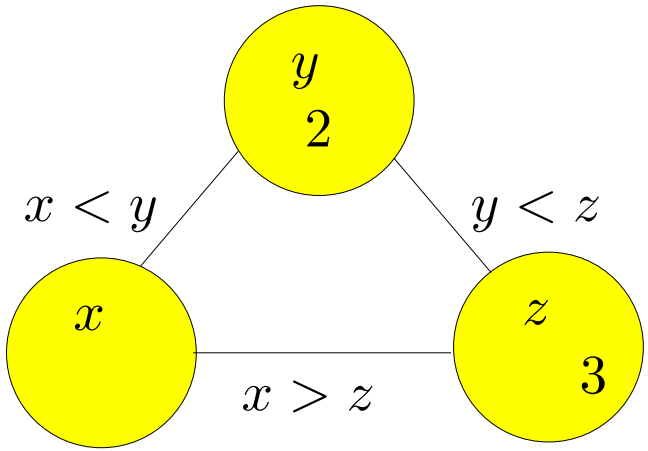
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y,\ y < z,\ x > z$



Arc Consistency

- Another example of enforcing AC.

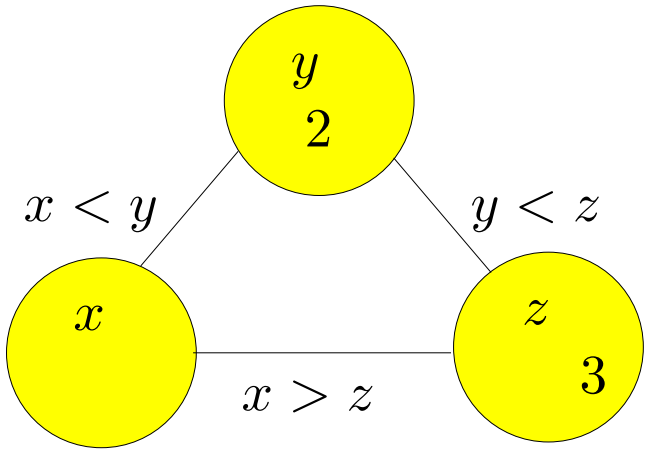
  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y,\ y < z,\ x > z$



Arc Consistency

- Another example of enforcing AC.

  Consider the CSP with variables x, y, z,
  domains $d_x = d_y = \{1, 2, 3\}$ and $d_z = \{0, 2, 3\}$,
  and constraints $x < y,\ y < z,\ x > z$



- The domain of x has become empty!
- So the CSP cannot have any solution

Arc Consistency

- If while enforcing AC some domain becomes empty,
  then we know the original CSP has no solution
- Else, when there are no more arc-inconsistent values,
  we have a smaller equivalent CSP (no solution is lost)
- Uniqueness: the order of removal of arc-inconsistent values is irrelevant
- Now let us see algorithms for effectively enforcing AC

Algorithms

AC-1: Revise(i, j)

The simplest algorithm to enforce AC is called AC-1

Based on function `Revise`(i, j),
which removes values from d_i without support in d_j.
Returns true if some value is removed

function `Revise`(i, j)
    change := false
    for each $a \in d_i$ **do**
        if $\forall_{b \in d_j} \neg c_{ij}(a, b)$ **then**
            change := true
            remove a **from** d_i
    return change

The time complexity of `Revise`(i, j) is $O(|d_i| \cdot |d_j|)$
(we assume that evaluating a binary constraint takes constant time)

AC-1

procedure AC-1(X, D, C)
    repeat
        change := $false$
        for each $c_{ij} \in C$ **do**
            change := change $\vee$ Revise(i, j) $\vee$ Revise(j, i)
    until $\neg$ change

- The time complexity of AC-1 is $O(e \cdot n \cdot m^3)$,
  with $n = |X|$, $m = \max_i\{|d_i|\}$ and $e = |C|$ (note $e = O(n^2)$)
- Whenever a value has been removed,
  the domain should be checked if empty (not done here for simplicity)

AC-3

- A more efficient algorithm is AC-3,
  which only revises constraints with a chance of filtering domains
- AC-3 uses a set of pairs Q s.t.
  if $(i, j) \in Q$ then we can't ensure that all values in d_i have support in x_j

procedure AC-3(X, D, C)
  $Q := \{(i, j), (j, i) \mid c_{ij} \in C\}$ // each pair is added twice
  while $Q \neq \emptyset$ **do**
    $(i, j) :=$ Fetch(Q) // selects and removes from Q
    if Revise(i, j) **then**
      $Q := Q \cup \{(k, i) \mid c_{ki} \in C, k \neq j\}$

- Space complexity: $O(e)$
- Time complexity: $O(e \cdot m^3)$

Complexity of AC-3

$Q := \{(i,j),(j,i) |\ \ c_{ij} \in C\}$ // each pair is added twice
while $Q \neq \emptyset$ **do**
  $(i,j) :=$ `Fetch`(Q) // selects and removes from Q
  if `Revise`(i,j) **then**
    $Q := Q \cup \{(k,i) \mid c_{ki} \in C, k \neq j\}$

- (i,j) is in Q because d_j has been pruned.
  Therefore, it will be in Q at most m times.
  Consequently, the loop iterates at most $O(e \cdot m)$ times
- Without the red part, the cost of AC-3 would be $O(e \cdot m^3)$
- For a given i, `Revise`(i,j) will be true at most m times.
  Aggregated cost of red part due to i is

$$O(|\{k \mid c_{ki} \in C\}| \cdot m) = O(\deg(i) \cdot m)$$

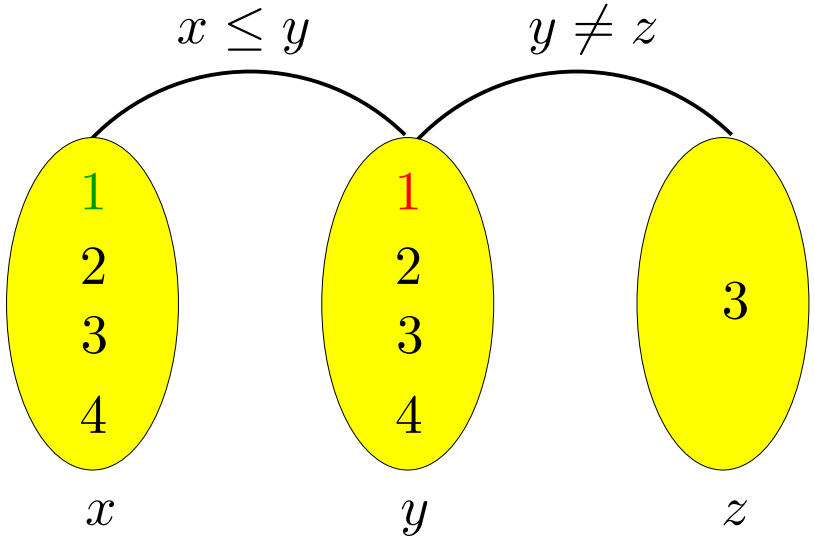
  So aggregated cost of red part due to all vars is $O(e \cdot m)$
- So the total cost in time is $O(e \cdot m^3)$

Example of AC-3

- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$

Example of AC-3

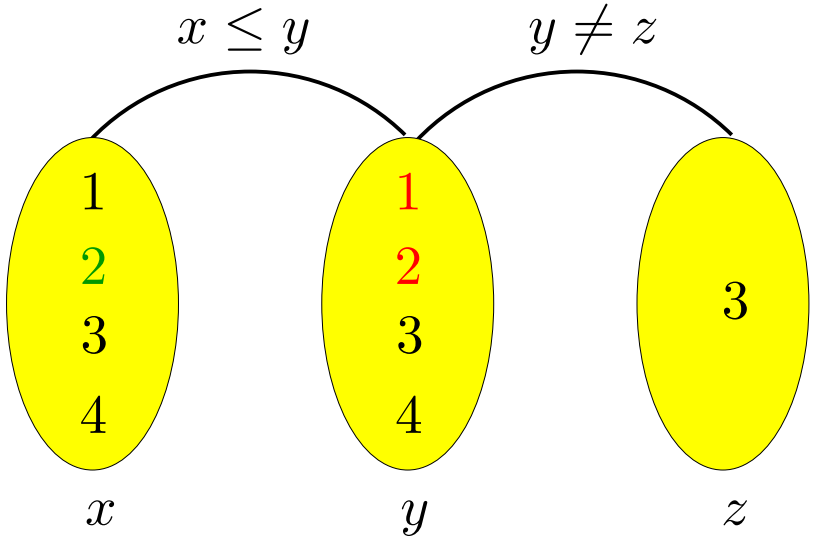
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(x, y)
  - Finding support for $(x, 1)$: 1

Example of AC-3

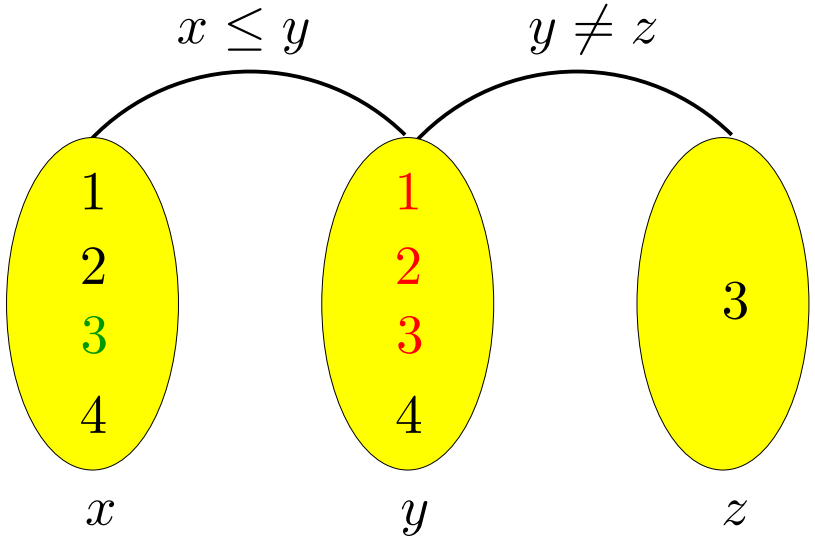
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(x, y)
  - Finding support for $(x, 2)$: 2

Example of AC-3

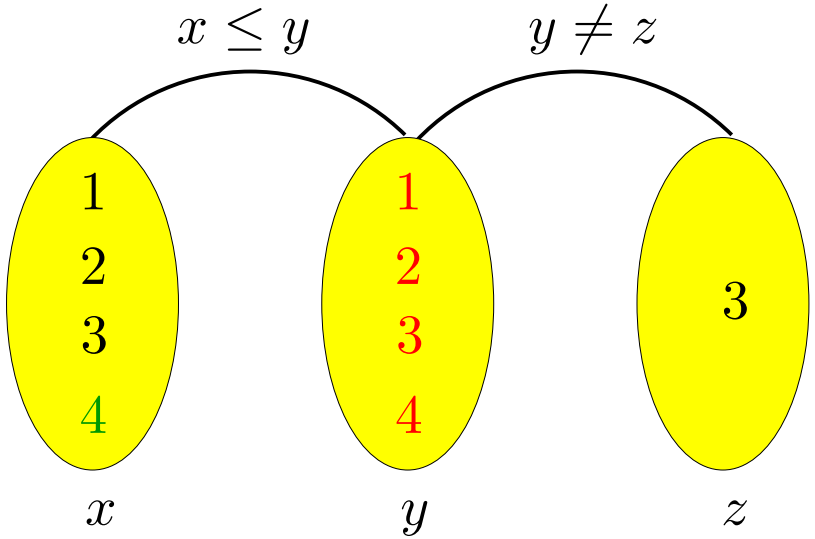
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(x, y)
  - Finding support for $(x, 3)$: 3

Example of AC-3

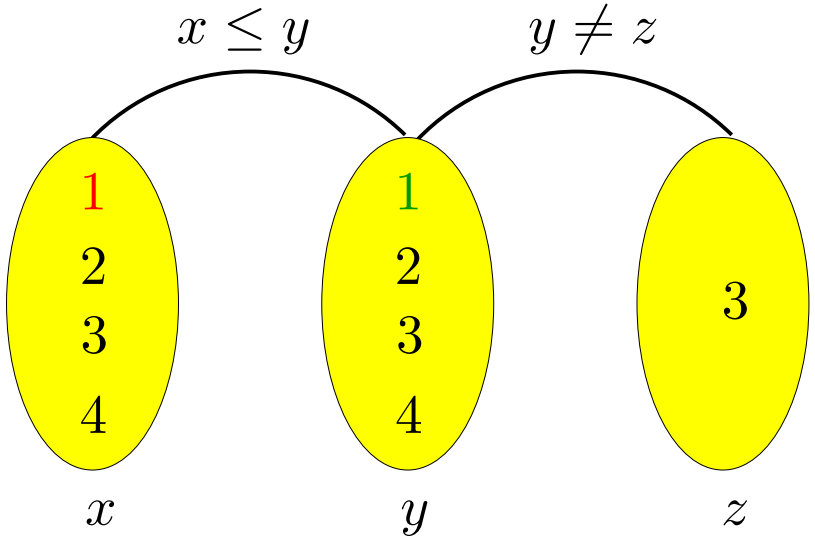
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(x, y)
  - Finding support for $(x, 4)$: 4

Example of AC-3

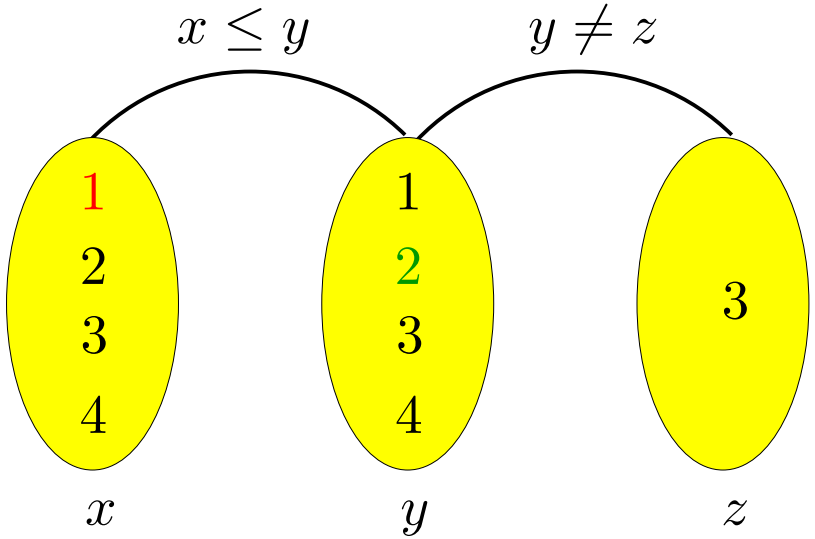
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(y, x)
  - Finding support for $(y, 1)$: 1

Example of AC-3

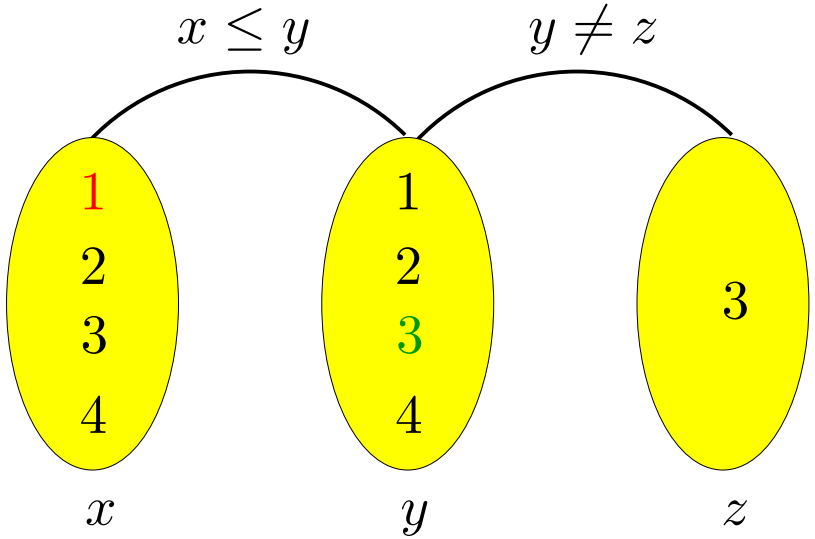
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(y, x)
  - Finding support for $(y, 2)$: 1

Example of AC-3

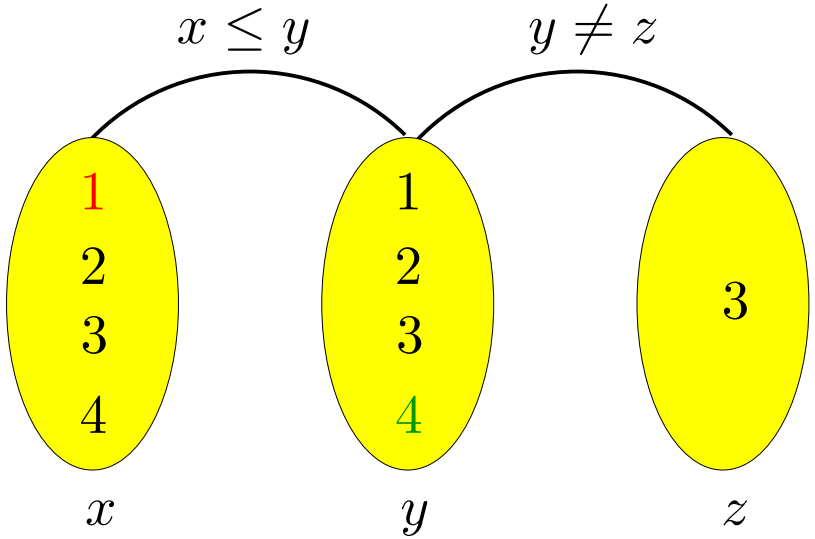
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(y, x)
  - Finding support for $(y, 3)$: 1

Example of AC-3

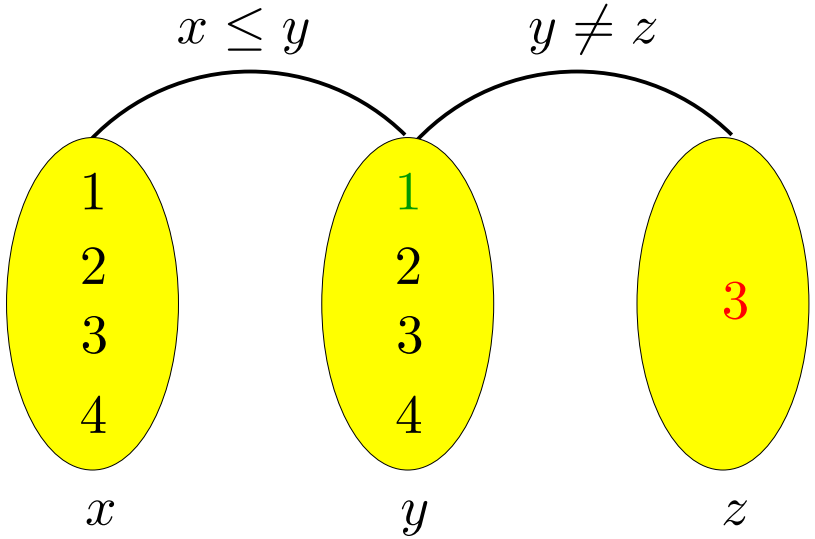
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(y, x)
  - Finding support for $(y, 4)$: 1

Example of AC-3

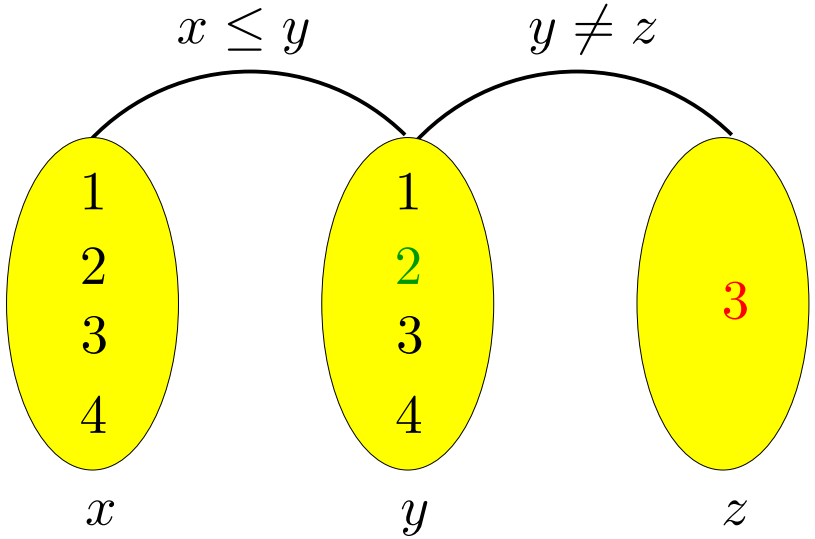
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(y, z)
  - Finding support for $(y, 1)$: 1

Example of AC-3

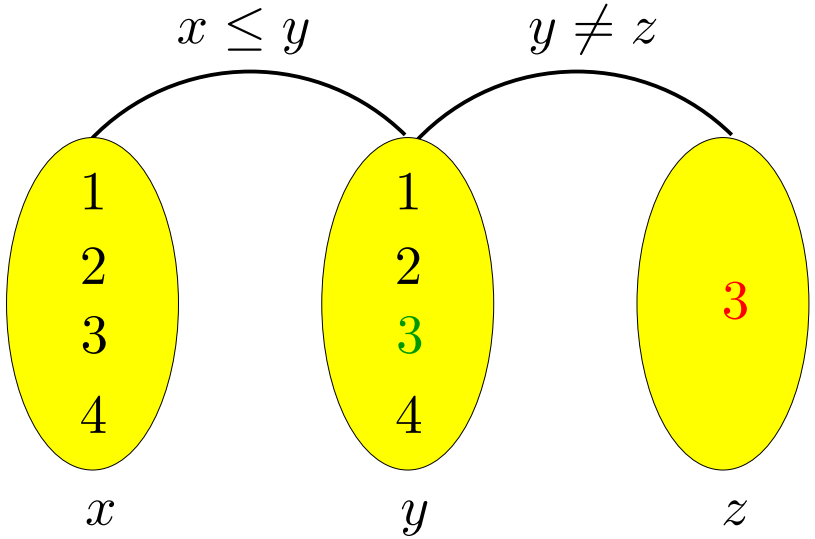
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(y, z)
  - Finding support for $(y, 2)$: 1

Example of AC-3

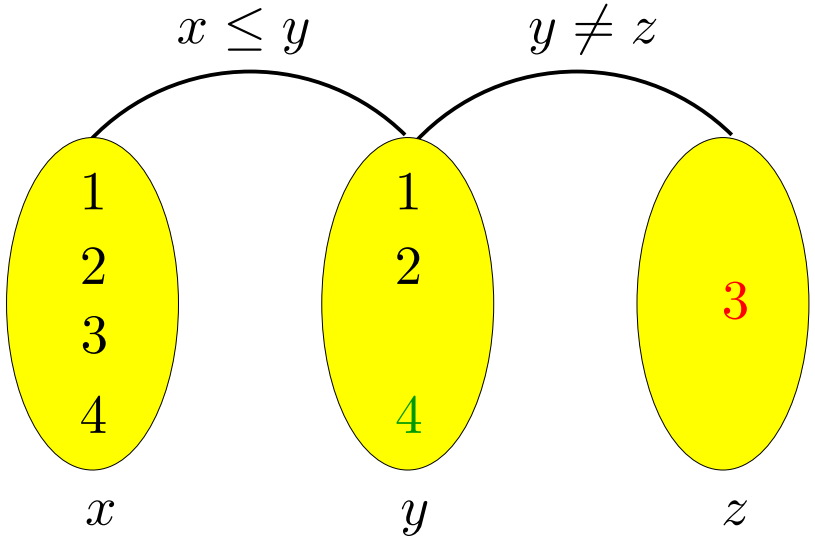
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(y, z)
  - Finding support for $(y, 3)$: 1

Example of AC-3

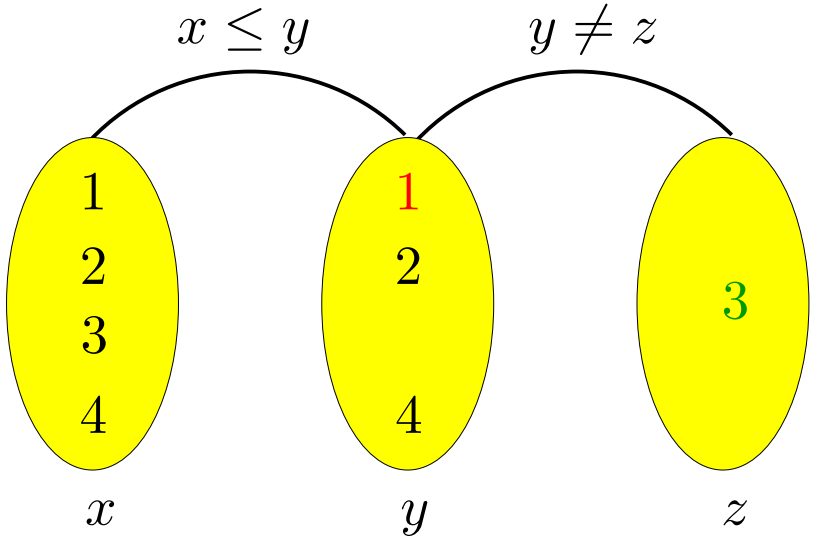
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(y, z)
  - Finding support for $(y, 4)$: 1

Example of AC-3

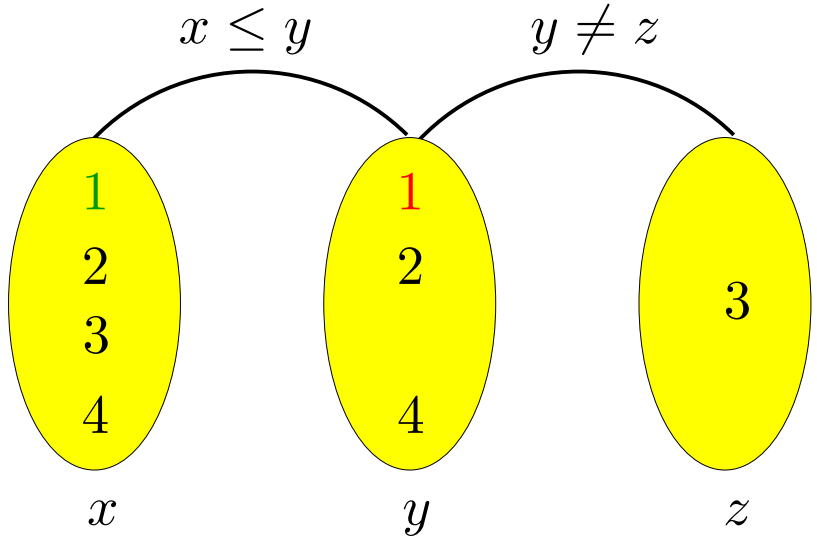
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(z, y)
  - Finding support for $(z, 3)$: 1

Example of AC-3

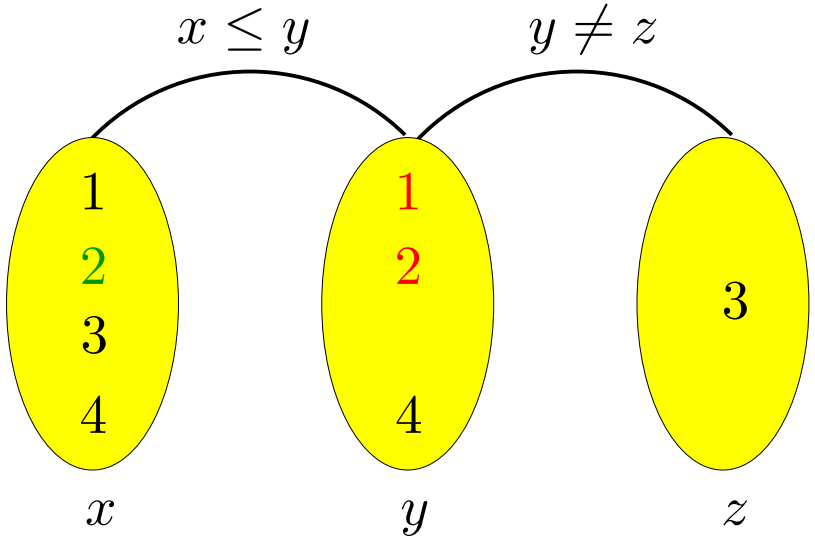
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(x, y)
  - Finding support for $(x, 1)$: 1

Example of AC-3

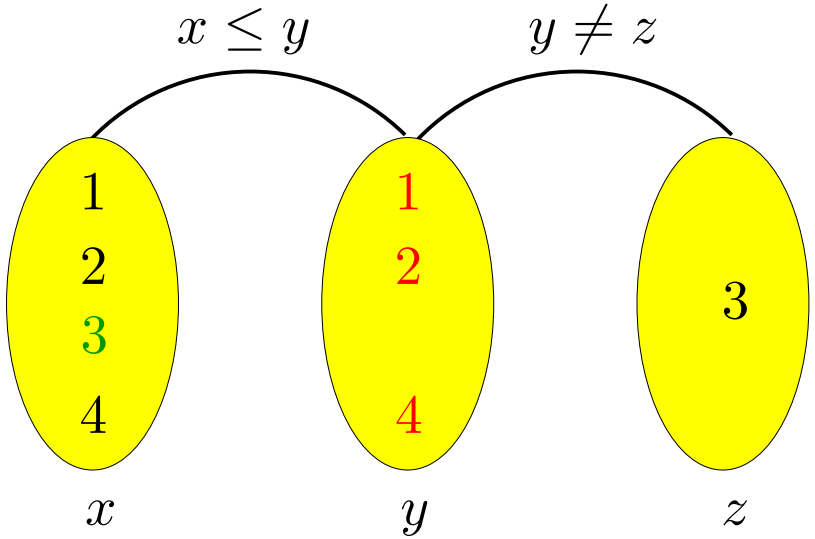
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(x, y)
  - Finding support for $(x, 2)$: 2

Example of AC-3

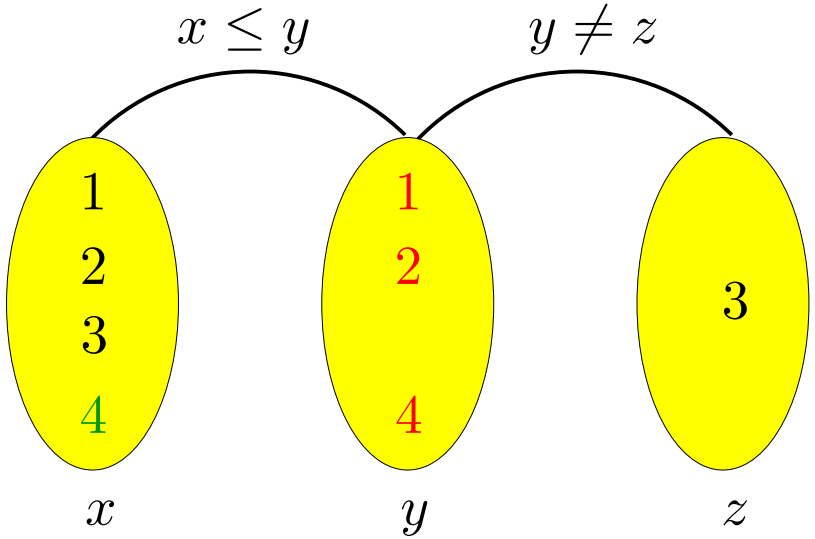
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(x, y)
  - Finding support for $(x, 3)$: 3

Example of AC-3

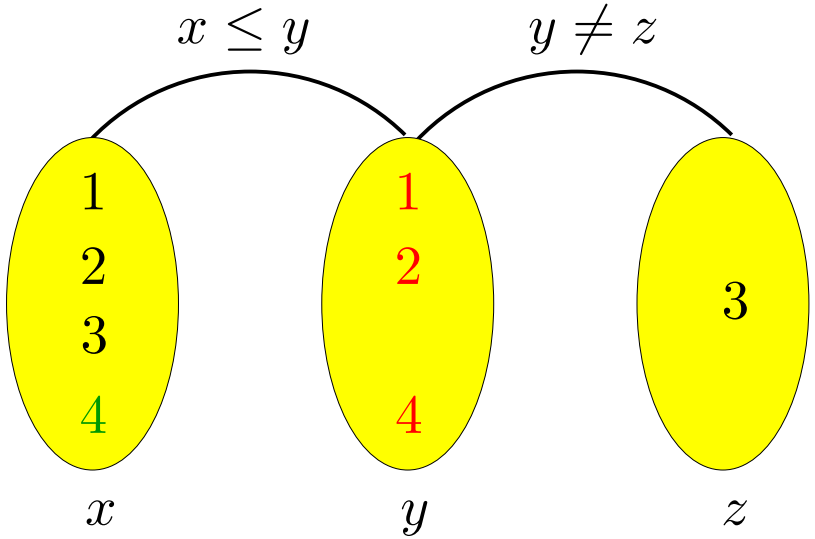
- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Let us count the number of constraint checks of `Revise`(x, y)
  - Finding support for $(x, 4)$: 3

Example of AC-3

- Let x, y, z be vars with domains $d_x = d_y = \{1, 2, 3, 4\}$, $d_z = \{3\}$, and $c_1 \equiv x \leq y$, $c_2 \equiv y \neq z$



- Altogether $10 + 4 + 4 + 1 + 9 = 28$ constraint checks
- From last 9, the only new check is $((x, 3), (y, 4))$!
- Still a lot of work is repeated over and over again

AC-4

- AC-4 is an even more efficient algorithm. It uses:
  - $N[i, a, j] =$ "number of supports that $a \in d_i$ has in d_j"
  - $S[j, b] =$ "set of pairs (i, a) supported by $b \in d_j$"
  - $(i, a) \in Q$ means that a has been pruned from d_i and its effect has not been updated on the N array yet

procedure AC-4(X, D, C)

// N is constructed full of 0's and S is constructed full of $\emptyset$'s

// Initialization phase

for each $c_{ij} \in C$, $a \in d_i$, $b \in d_j$ **such that** $c_{ij}(a, b)$ **do**

// Value b in d_j is a support for value $a \in d_i$

$N[i, a, j] + +$

$S[j, b] := S[j, b] \cup \{(i, a)\}$

for each $c_{ij} \in C$, $a \in d_i$ **such that** $N[i, a, j] = 0$ **do**

remove a **from** d_i

$Q := Q \cup (i, a)$

AC-4

...
// Propagation phase
while $Q \neq \emptyset$ **do**
    $(j, b) := \texttt{Fetch}(Q)$
    for each $(i, a) \in S[j, b]$ **such that** $a \in d_i$ **do**
        $N[i, a, j] - -$
        if $N[i, a, j] = 0$ **then**
            remove a **from** d_i
            $Q := Q \cup (i, a)$

- Time complexity of AC-4: $O(e \cdot m^2)$ (provable optimal!)
  - the initialization phase has cost $O(e \cdot m^2)$
  - the propagation phase has cost $O(e \cdot m^2)$
- Space complexity of AC-4: $O(e \cdot m^2)$

Example of AC-4

- The only counter equal to zero is $N[y, 3, z]$
- $(y, 3)$ is removed, propagation loop starts with $(y, 3)$ in Q
- When $(y, 3)$ is picked, traverse $S[y, 3] = \{(x, 1), (x, 2), (x, 3)\}$
  - $N[x, 1, y], N[x, 2, y], N[x, 3, y]$ are decremented
- No counter becomes zero, so nothing is added to Q
- Propagation did not require any extra constraint check!
- However
  - Space complexity is very high
  - The initialization phase can be prohibitive
    (AC-4 has optimal worst-case complexity ...
    but always reaches this worst case)

AC-6

- Motivation: keep the optimal worst-case of AC-4 but instead of counting all supports that a value has, just ensure that there is at least one
- AC-6 keeps the smallest support for each (x_i, a) on c_{ij}

AC-6

- Motivation: keep the optimal worst-case of AC-4
  but instead of counting all supports that a value has,
  just ensure that there is at least one
- AC-6 keeps the smallest support for each (x_i, a) on c_{ij}

- *Initialization phase*: cheaper than in AC-4
- *Propagation phase*: if value b of var x_j is removed
  - if it is not the current support of value a of var x_i:
    no work due to constraint c_{ij} has to be done
  - if it is the current support of value a of var x_i:
    a new support is sought,
    but starting from next value of b in d_j instead of $\min\{d_j\}$

AC-6

- Motivation: keep the optimal worst-case of AC-4
  but instead of counting all supports that a value has,
  just ensure that there is at least one
- AC-6 keeps the smallest support for each (x_i, a) on c_{ij}

- *Initialization phase*: cheaper than in AC-4
- *Propagation phase*: if value b of var x_j is removed
  - if it is not the current support of value a of var x_i:
    no work due to constraint c_{ij} has to be done
  - if it is the current support of value a of var x_i:
    a new support is sought,
    but starting from next value of b in d_j instead of $\min\{d_j\}$

- The algorithm uses:
  $S[j, b] =$ "set of (i, a) s.t. b is current support of value a of x_i wrt. x_j"
  $Q =$ "set of (i, a) s.t. a has been pruned from d_i but
  its effect has not been updated on S yet"

AC-6

procedure AC-6(X, D, C)

$Q := \emptyset$; $S[j, b] := \emptyset$, $\forall x_j \in X$, $\forall b \in d_j$

// Initialization phase

for each $x_i \in X$, $c_{ij} \in C$, $a \in d_i$ **do**

$b :=$ smallest value in d_j s.t. $c_{ij}(a, b)$

if $b \neq \perp$ **then** add (i, a) to $S[j, b]$

else remove a from d_i and add (i, a) to Q

// Propagation phase

while $Q \neq \emptyset$ **do**

$(j, b) :=$ Fetch(Q)

for each $(i, a) \in S[j, b]$ **such that** $a \in d_i$ **do**

$c :=$ smallest value $b' \in d_j$ s.t. $b' > b \wedge c_{ij}(a, b')$

if $c \neq \perp$ **then** add (i, a) to $S[j, c]$

else remove a from d_i and add (i, a) to Q

- Time complexity: $O(e \cdot m^2)$
- Space complexity: $O(e \cdot m)$

Conclusion

Conclusion

- Other types of consistency:
  - Weaker than AC
    - Directional AC (DAC)
    - Bounds Consistency (BC)
  - Stronger than AC
    - Singleton AC (SAC)
    - Neighborhood Inverse Consistency (NIC)
- Advantages:
  - Many optimized algorithms
  - Time complexity is reduced